

UNIVERSITATEA TEHNICĂ "GHEORGHE ASACHI" DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

VARIANTĂ PRELIMINARĂ A LUCRĂRII DE DIPLOMĂ
Raport Semestrul II

**Sistem IoT Distribuit pentru Detectarea Timpurie
a Incendiilor cu Rutare Adaptivă și
Monitorizare Web în Timp Real**

Student: Ionescu Vlad-Gabriel
Îndrumător: Doctor Inginer Vieriu George-Emil

Iunie 2026

Rezumat

Lucrarea de față prezintă proiectarea și implementarea unui sistem IoT distribuit destinat detectării timpurii a incendiilor în spații industriale sau publice. Sistemul este construit pe o rețea de senzori fără fir (WSN – Wireless Sensor Network) formată din patru noduri ESP32 care colectează date de temperatură, umiditate, presiune atmosferică și concentrație de gaz combustibil. Datele sunt transmise prin protocolul ESP-NOW într-o topologie mesh către un gateway central, conectat printr-o interfață serială USB la un server Raspberry Pi.

Rutarea mesajelor în rețeaua mesh se realizează printr-un algoritm adaptiv bazat pe Dijkstra, cu funcție de cost multi-criteriu ce include numărul de hop-uri, calitatea semnalului radio (RSSI), nivelul bateriei nodurilor intermediare și gradul de congestie. Coeficienții funcției de cost sunt modificați dinamic în funcție de starea de alertă a rețelei, prioritizând latența minimă în situații de urgență.

Un motor de evaluare a riscului calculează un scor 0–100 pentru fiecare nod pe baza a cinci componente: temperatura măsurată, concentrația de gaz, tendința de creștere termică, confirmarea de la noduri vecine și calitatea comunicației. Pragurile de clasificare sunt configurabile dinamic de către operator.

Componenta server rulează pe Raspberry Pi și este implementată în Python/FastAPI cu comunicație WebSocket în timp real. Interfața web accesibilă de la distanță afișează harta planului etajului cu starea nodurilor, grafice live ale parametrilor senzorilor, jurnal de evenimente și panou de control pentru comenzi.

Sistemul funcționează complet cu date simulate pe infrastructura Raspberry Pi, urmând integrarea hardware a senzorilor fizici în etapa următoare.

1. Introducere

1.1 Contextul General și Motivarea Temei

Incendiile reprezintă una dintre cele mai grave amenințări la adresa vieții umane și a infrastructurii construite. Conform datelor Inspectoratului General pentru Situații de Urgență (IGSU), în România se înregistrează anual peste 25.000 de incendii, cu sute de victime și pagube materiale de ordinul miliardelor de lei. La nivel european, statisticile CTIF (Comitetul Tehnic Internațional pentru Prevenirea și Stingerea Incendiilor) arată că aproximativ 70% din decesele cauzate de incendii survin în primele 3 minute de la izbucnire, perioadă în care detectarea și evacuarea sunt esențiale [1].

Sistemele tradiționale de detecție a incendiilor — detectoare de fum punctuale, termodetectoare cu fir sau centrale de alarmă convenționale — prezintă limitări semnificative în

contextul actual. Acoperirea spațială este limitată de raza fiecărui detector, instalarea necesită infrastructură de cablare costisitoare, iar reconfigurarea topologiei impune intervenție fizică. În plus, aceste sisteme nu oferă capacități de localizare precisă a sursei incendiului și sunt vulnerabile la alarme false generate de un singur senzor.

Avansul tehnologic din domeniul IoT (Internet of Things) și al rețelelor de senzori fără fir (WSN – Wireless Sensor Network) oferă o alternativă scalabilă, flexibilă și eficientă din punct de vedere al costurilor. Nodurile senzori autonome, cu alimentare din baterie și comunicație wireless, pot acoperi suprafețe mari fără infrastructură de cablare, pot fi redistribuite ușor și pot transmite date în timp real către o platformă centrală de monitorizare accesibilă de oriunde.

1.2 Probleme Deschise și Motivarea Abordării Propuse

Implementarea unui sistem WSN pentru detecția incendiilor ridică mai multe provocări tehnice care nu sunt complet rezolvate în soluțiile existente:

- **Fiabilitatea rutării în rețele mesh:** Dacă un nod intermediar cade sau are baterie scăzută, mesajele de alertă pot fi pierdute sau întârziate. Sistemele existente utilizează adesea rutare statică, fără adaptare la condițiile dinamice ale rețelei.
- **Reducerea alarmelor false:** Un singur senzor de temperatură sau fum poate genera alarme eronate din cauza variațiilor normale ale mediului (gătit, praf, umiditate ridicată). Corelarea datelor din multiple surse și confirmarea de la noduri vecine sunt necesare.
- **Scalabilitatea platformei software:** Soluțiile existente sunt adesea proprietare, cu interfețe web limitate și fără acces remote standardizat. Integrarea cu infrastructuri cloud sau sisteme de alertare externă este dificilă.
- **Adaptabilitatea la starea de urgență:** În situații de incendiu, prioritățile rețelei trebuie să se schimbe automat: latența minimă devine critică, chiar cu costul consumului energetic mai mare.

Lucrarea propune o soluție care adresează toate aceste provocări printr-o arhitectură originală: algoritm de rutare adaptivă multi-criteriu, motor de risc cu confirmare multi-senzor, și platformă software deschisă cu acces web în timp real.

1.3 Obiectivele Lucrării

Obiectivul general al lucrării este proiectarea și implementarea unui sistem IoT distribuit funcțional pentru detectarea timpurie a incendiilor, cu monitorizare web în timp real și rutare adaptivă a mesajelor în rețeaua mesh.

Obiectivele specifice sunt:

- Implementarea unei rețele mesh de 4 noduri senzori ESP32 cu comunicație ESP-NOW și firmware complet funcțional.
- Proiectarea și implementarea unui algoritm de rutare adaptivă bazat pe Dijkstra cu funcție de cost multi-criteriu.
- Dezvoltarea unui motor de evaluare a riscului multi-parametric cu scor 0–100 și praguri configurabile dinamic.
- Implementarea unui backend FastAPI cu comunicație WebSocket în timp real și persistență SQLite.
- Crearea unei interfețe web de monitorizare accesibile remote, cu harta planului etajului, grafice live și panou de control.
- Validarea funcționării sistemului prin testare extensivă în modul de simulare.

1.4 Structura Lucrării

Lucrarea este organizată în cinci capitole principale. Capitolul 2 prezintă fundamentarea teoretică și analiza soluțiilor similare existente. Capitolul 3 descrie în detaliu soluția propusă, arhitectura sistemului și implementarea componentelor principale. Capitolul 4 prezintă activitățile desfășurate și rezultatele obținute până în prezent. Capitolul 5 sintetizează concluziile și direcțiile de dezvoltare viitoare.

2. Fundamentare Teoretică și Soluții Similare

2.1 Rețele de Senzori Fără Fir (WSN)

O rețea WSN (Wireless Sensor Network) este formată din noduri senzori cu resurse limitate distribuite fizic într-un spațiu monitorizat. Fiecare nod integrează un microcontroler, unul sau mai mulți senzori, un modul radio pentru comunicație fără fir și o sursă de alimentare (baterie sau energie recoltată din mediu) [2]. Nodurile comunică între ele și cu un gateway central (sink node) care agregază datele și le transmite către un sistem de procesare superioară.

Topologiile utilizate în WSN includ:

- **Stea (Star):** Fiecare nod senzor comunică direct cu gateway-ul. Simplă dar limitată ca rază de acoperire — nodurile îndepărtate sau obstrucționate nu pot comunica.
- **Arbore (Tree):** Nodurile sunt organizate ierarhic. Oferă acoperire mai mare dar fără redundanță — dacă un nod intermediar cade, întreg sub-arborele devine inaccesibil.
- **Mesh:** Fiecare nod poate comunica cu oricare vecin. Oferă redundanță maximă și auto-vindecare la căderea unui nod. Este topologia preferată în aplicații critice precum detecția incendiilor.

- **Hibrid:** Combinație de topologii. De exemplu, clustere stea interconectate mesh. Echilibrează consumul energetic cu redundanța.

Provocările principale ale WSN includ: managementul energiei (nodurile au baterii limitate), fiabilitatea comunicației (interferențe radio, obstacole fizice), latența end-to-end și securitatea datelor transmise wireless.

2.2 Protocoale de Comunicație pentru WSN

Alegerea protocolului de comunicație fără fir este critică pentru performanța unui sistem WSN. Principalele protocoale utilizate în aplicații IoT/WSN sunt:

- **Zigbee (IEEE 802.15.4):** Protocol mesh standardizat, frecvență 2.4 GHz, viteză 250 kbps, distanță 10–100m. Consum energetic redus (hibernare sub 1μA). Larg utilizat în automatizări industriale și sisteme de securitate. Necesită hardware dedicat (modul Zigbee).
- **Z-Wave:** Protocol proprietar (sub 1GHz), interferențe reduse cu WiFi. Utilizat predominant în automatizări rezidențiale. Ecosistem limitat față de Zigbee.
- **LoRa/LoRaWAN:** Distanțe foarte mari (km), consum extrem de redus, viteză mică (0.3–50 kbps). Ideal pentru monitorizare forestieră sau agricolă, nu pentru sisteme cu latență critică de câteva secunde.
- **ESP-NOW (Espressif):** Protocol peer-to-peer proprietar bazat pe WiFi (802.11), fără necesitatea unui router WiFi. Latență sub 2ms, viteză până la 1 Mbps, distanță 200m în câmp deschis. Suportat nativ de ESP32 și ESP8266 [3].
- **Thread/Matter:** Standard IoT modern bazat pe IPv6 mesh (802.15.4). Adoptat de Apple, Google, Amazon pentru smart home. Necesită hardware compatibil.

Pentru sistemul propus, ESP-NOW a fost ales datorită compatibilității native cu platforma ESP32 (fără hardware adițional), latenței reduse critice în situații de urgență, și simplității implementării mesh ad-hoc fără configurare de router.

2.3 Microcontrollerul ESP32

ESP32 (Espressif Systems) este un System-on-Chip (SoC) cu procesor dual-core Xtensa LX6 la 240 MHz, memorie flash de 4–16 MB, SRAM 520 KB, WiFi 802.11 b/g/n și Bluetooth 4.2/5.0 integrate. Caracteristicile relevante pentru proiect sunt:

- ADC 12-bit pe 18 canale – citire senzori analogici (gaz MQ) cu rezoluție 0–4095
- I2C hardware – comunicație cu senzorul digital BME280 (temperatură, umiditate, presiune)
- Deep-sleep cu consum sub 10μA – prelungire durată baterie între citiri
- ESP-NOW integrat – comunicație mesh fără router WiFi, latență < 2ms

- Dual-core – un core pentru achiziție date, altul pentru comunicație
- Preț accesibil – module ESP32-WROOM sub 5 EUR, cost total per nod sub 25 EUR

2.4 Senzori Utilizați

2.4.1 Senzorul BME280 (Bosch)

BME280 este un senzor digital de înaltă precizie care măsoară simultan temperatura ($\pm 0.5^{\circ}\text{C}$), umiditatea relativă ($\pm 3\%$) și presiunea atmosferică ($\pm 1 \text{ hPa}$). Comunicația se realizează prin I2C (adresă 0x76 sau 0x77) sau SPI. Consumul în modul normal este de $3.6\mu\text{A}$, extrem de eficient energetic. Precizia ridicată și consumul redus îl fac alegerea standard pentru stații meteorologice și sisteme IoT [4].

În contextul detecției incendiilor, BME280 furnizează datele primare de temperatură (indicator principal al incendiului), umiditate (scădere bruscă asociată incendiului) și presiune (variații asociate jeturilor de aer cald).

2.4.2 Senzorul de Gaz MQ (seria MQ-2/MQ-5/MQ-135)

Senzorii din seria MQ sunt senzori electrochimici cu element sensibil SnO_2 (dioxid de staniu). Rezistența elementului sensibil variază în prezența gazelor combustibile (metan, propan, hidrogen, monoxid de carbon, fum). Semnalul de ieșire este analogic (0–5V), citit prin ADC-ul ESP32 cu rezoluție 12-bit (0–4095). Dezavantajul principal este necesitatea unui timp de încălzire de 30–60 secunde la pornire și o perioadă de calibrare de 24 ore la prima utilizare pentru stabilizarea valorii de referință [5].

2.5 Algoritmi de Rutare în Rețele Mesh

Rutarea eficientă a mesajelor într-o rețea mesh WSN este esențială pentru fiabilitatea sistemului. Principalele categorii de algoritmi sunt:

- **Rutare proactivă (table-driven):** Fiecare nod menține o tabelă de rutare actualizată continuu prin schimb periodic de informații cu vecinii. OLSR (Optimized Link State Routing) și DSDV sunt exemple reprezentative. Avantaj: latență mică la trimiterea primului pachet. Dezavantaj: overhead de control ridicat.
- **Rutare reactivă (on-demand):** Rutele sunt calculate doar la nevoie (AODV, DSR). Avantaj: overhead redus. Dezavantaj: latență la primul pachet din cauza descoperirii rutei.
- **Rutare bazată pe gradient:** Mesajele sunt rutate urmând gradientul unui câmp scalar (exemplu: calitatea semnalului). Simplă dar suboptimală.
- **Rutare bazată pe Dijkstra:** Algoritmul Dijkstra calculează drumul minim cost între oricare două noduri din graf. Utilizat în protocoale proactive (OSPF în rețele IP). Optim din punct de vedere al funcției de cost, cu complexitate $O((V+E) \cdot \log V)$ [6].

În sistemul propus, algoritmul Dijkstra rulează pe fiecare nod senzor înainte de trimiterea unui pachet, utilizând tabela de vecini statică (configurată la instalare) îmbunătățită cu date live despre starea nodurilor (RSSI măsurat, nivel baterie, status online/offline). Funcția de cost a unui link este:

$$C(\text{link}) = \alpha \cdot \text{hop} + \beta \cdot \text{pen_RSSI} + \gamma \cdot \text{pen_baterie} + \delta \cdot \text{pen_congestie}$$

unde α , β , γ , δ sunt coeficienți de ponderare cu sumă 1, configurați în două seturi: modul NORMAL (prioritate baterie: $\alpha=0.30$, $\beta=0.20$, $\gamma=0.40$, $\delta=0.10$) și modul ALERTĂ (prioritate latență: $\alpha=0.50$, $\beta=0.40$, $\gamma=0.05$, $\delta=0.05$).

2.6 Sisteme Existente de Detectare a Incendiilor

Soluțiile comerciale actuale pentru detecția incendiilor se împart în două categorii:

2.6.1 Sisteme Convenționale Cablate

Sistemele precum Bosch FPA-5000, Hochiki ESP, Siemens Cerberus PRO sunt soluții centralizate cu cabluri dedicate, certificate conform EN 54 (standard european pentru sisteme de detecție și alarmare la incendiu). Principalele caracteristici sunt:

- Fiabilitate ridicată datorită cablurilor dedicate (imune la interferențe radio)
- Certificate conform standardelor europene EN 54-2, EN 54-4
- Cost instalare ridicat: 10.000–50.000 EUR pentru o clădire medie
- Reconfigurare dificilă – necesită intervenție fizică la cabluri
- Fără capabilități de acces remote sau monitorizare web nativă

2.6.2 Soluții WSN din Literatura de Specialitate

Cercetările recente propun diverse sisteme WSN pentru detecția incendiilor. Sharma et al. [7] propun o rețea Zigbee cu senzori de temperatură și fum pentru depozite industriale, demonstrând îmbunătățiri ale timpului de detecție față de sistemele convenționale, dar fără algoritm de rutare adaptivă. Zhang et al. [8] introduc un algoritm de fuziune a datelor din multiple noduri pentru reducerea alarmelor false, prin corelarea datelor din mai multe noduri. Caballero et al. [9] prezintă un sistem bazat pe machine learning pentru detecția incendiilor forestiere cu noduri LoRa.

Față de aceste lucrări, soluția propusă în prezenta lucrare aduce contribuții originale:

- Algoritm de rutare adaptivă cu schimbare dinamică a funcției de cost în funcție de starea de alertă a rețelei
- Motor de risc multi-parametric cu confirmare de la noduri vecine, reducând alarmele false

- Platformă software completă open-source cu interfață web accesibilă remote
- Arhitectură cu mod de mentenanță distinct pentru noduri în service, fără alarme false

3. Soluția Propusă

3.1 Arhitectura Generală a Sistemului

Sistemul propus are o arhitectură pe trei niveluri bine definite, fiecare cu responsabilități clare și protocoale de comunicație standardizate:

- **Nivelul 1 – Percepție (Noduri Senzori ESP32):** Patru noduri ESP32, fiecare echipat cu senzor BME280 și senzor MQ de gaz, distribuiți fizic în zonele monitorizate. Fiecare nod rulează firmware C++ care achiziționează date, calculează scorul de risc local și determină ruta optimă prin algoritmul Dijkstra.
- **Nivelul 2 – Transport (Gateway ESP32 + Serial):** Un al cincilea ESP32, configurat ca gateway, recepționează pachetele ESP-NOW de la toate nodurile senzori și le retransmite prin interfața serială USB (115200 baud) în format JSON către Raspberry Pi.
- **Nivelul 3 – Aplicație (Raspberry Pi + Web):** Raspberry Pi rulează backend-ul Python/FastAPI care procesează pachetele, actualizează starea nodurilor, persistă evenimentele în SQLite și bcastează datele în timp real prin WebSocket la toți clienții web conectați.

Un pachet de date parcurge drumul: nodul senzor calculează ruta prin Dijkstra, transmite prin ESP-NOW la gateway, care retransmite prin USB serial în format JSON la backend-ul FastAPI. Acesta procesează datele prin motorul de risc și le bcastează prin WebSocket la clienții web conectați. Latența estimată end-to-end este sub 500ms.

3.2 Topologia Rețelei Mesh

Rețeaua mesh este formată din 4 noduri senzori (N1–N4) și un nod gateway (GW), plasate conform planului fizic al spațiului monitorizat. Zonele acoperite sunt:

- **N1 – Zona Intrare:** monitorizare intrări principale, risc moderat
- **N2 – Zona Depozit:** monitorizare depozit materiale, risc ridicat, conectat direct la gateway
- **N3 – Zona Parcare:** monitorizare parcare subterană, risc gaz ridicat
- **N4 – Zona Tablou Electric:** monitorizare tablou electric, risc incendiu electric
- **GW – Camera Tehnică:** gateway central, conectat la Raspberry Pi via USB

Tabela de vecini configurată (cu valori RSSI de bază în dBm) este:

- N1: vecini N2 (–55 dBm), N4 (–68 dBm)
- N2: vecini N1 (–55 dBm), N3 (–62 dBm), GW (–50 dBm)
- N3: vecini N2 (–62 dBm), N4 (–65 dBm)
- N4: vecini N1 (–68 dBm), N3 (–65 dBm)
- GW: vecini N2 (–50 dBm)

Ruta primară a fiecărui nod senzor spre gateway (modul normal) este calculată prin algoritmul Dijkstra cu coeficienți normali. De exemplu, N1 rutează prin N2→GW, N3 rutează prin N2→GW, N4 rutează prin N3→N2→GW sau N1→N2→GW (ales dinamic în funcție de stare). Dacă N2 devine offline sau are RSSI slab, ruta se adaptează automat la traseul alternativ.

3.3 Firmware-ul Nodurilor Senzori

3.3.1 Structura Firmware-ului

Firmware-ul nodurilor senzori este implementat în C++ cu framework Arduino prin PlatformIO, organizat în module cu responsabilități clare:

- **sensors.cpp/h:** inițializare BME280 și senzor MQ, citire date, clasificare nivel gaz în 5 trepte (normal/scăzut/mediu/ridicat/critic)
- **routing.cpp/h:** algoritmul Dijkstra, calculul funcției de cost, determinarea rutei optime spre gateway
- **communication.cpp/h:** inițializare ESP-NOW, înregistrare peer-uri, trimitere/recepție pachete, gestionare ACK
- **commands.cpp/h:** procesare comenzi primite de la gateway (mute buzzer, reset alertă, calibrare, mentenanță)
- **config.h:** parametri configurabili: NODE_ID, MAC gateway, pini hardware, praguri gaz, intervale de timp
- **main.cpp:** loop principal: citire senzori → calcul risc → rutare → transmitere → procesare comenzi

3.3.2 Ciclul Principal de Operare

La fiecare ciclu de operare (implicit 3 secunde, configurabil), fiecare nod execută secvența:

- Citire senzori: temperatură, umiditate, presiune din BME280; valoare ADC brută din senzorul MQ
- Clasificare nivel gaz în 5 trepte pe baza pragurilor configurabile (normal_max, low_max, medium_max, high_max, critical_min)
- Calcul scor de risc 0–100 din 5 componente: temperatură, gaz, tendință termică, vecini în alertă, calitate comunicație

- Determinare stare nod: NORMAL (scor <40), WARNING (40–69), ALERT (≥ 70)
- Calcul rută optimă prin Dijkstra, adaptat la starea curentă (normal vs. alertă)
- Construire pachet JSON și transmitere ESP-NOW pe ruta calculată
- Activare buzzer local dacă starea este ALERT și pragul de buzzer este activat

3.4 Algoritmul de Rutare Adaptivă

Algoritmul Dijkstra implementat pe nodurile senzori calculează drumul de cost minim de la nodul curent la gateway (nodul 0), utilizând un graf orientat ponderat cu arcele reprezentând link-urile de comunicație ESP-NOW.

Funcția de cost pentru un arc (link) între nodurile u și v este:

- **Penalizare RSSI:** $\text{pen_RSSI} = \max(0, \min(1, (-40 - \text{RSSI_live}) / 60))$. RSSI ideal = -40 dBm, range normalizare = 60 dBm. Cu cât semnalul este mai slab (RSSI mai negativ), cu atât penalizarea este mai mare.
- **Penalizare baterie:** $\text{pen_baterie} = (100 - \text{baterie\%}) / 100$. Nodurile cu baterie aproape epuizată sunt evitate în modul normal pentru a prelungi durata de viață a rețelei.
- **Penalizare congestie:** $\text{pen_congestie} = \text{rata_erori}$ normalizată 0–1. Evitarea nodurilor cu rată mare de pierdere de pachete.
- **Cost hop:** Componentă fixă 1.0 per hop, ponderată cu α . În modul alertă ($\alpha=0.50$), latența minimă domină — se preferă rute directe chiar dacă trec prin noduri cu baterie scăzută.

Comutarea între seturi de coeficienți (normal/alertă) se face automat când scorul de risc depășește pragul de 70 (stare ALERT). Revenirea la coeficienții normali se face când scorul scade sub 40 pentru 5 citiri consecutive.

3.5 Motorul de Evaluare a Riscului

Motorul de evaluare a riscului este o componentă critică a sistemului, implementată atât pe nodurile senzori (calcul local pentru rutare) cât și pe backend (recalcul centralizat pentru consistență cu pragurile configurate de operator). Scorul final 0–100 este suma a cinci componente independente:

- **Scor temperatură (0–25 puncte):** Liniar de la 0 la 10 puncte între 20°C și pragul warning (45°C). Liniar de la 10 la 25 puncte între 45°C și pragul alert (65°C). Maxim 25 puncte peste 65°C.
- **Scor gaz (0–30 puncte):** Bazat pe nivelul clasificat al gazului: GAS_NORMAL=0, GAS_LOW=8, GAS_MEDIUM=15, GAS_HIGH=22, GAS_CRITICAL=30. Clasificarea respectă pragurile configurabile ale operatorului.

- **Scor tendință termică (0–20 puncte):** Calculat din istoricul ultimelor 5 citiri de temperatură. Creștere $<2^{\circ}\text{C}$ =0 puncte, $2-5^{\circ}\text{C}$ =5 puncte, $5-10^{\circ}\text{C}$ =12 puncte, $>10^{\circ}\text{C}$ =20 puncte. Detectează creșteri rapide caracteristice incendiilor.
- **Scor confirmare vecini (0–15 puncte):** 0 vecini în alertă/warning = 0 puncte. 1 vecin în alertă/warning = 7 puncte. ≥ 2 vecini în alertă/warning = 15 puncte. Componenta anti-alarme-false: confirmarea din noduri independente crește credibilitatea.
- **Scor comunicație (0–10 puncte):** Baterie $<20\%$ = +5 puncte (semnal că nodul poate fi compromis). RSSI <-80 dBm = +5 puncte (semnal slab, posibile pierderi de pachete).

Pragurile de clasificare finală sunt: scor 0–39 = NORMAL (verde), 40–69 = WARNING (portocaliu), 70–100 = ALERT (roșu). Aceste praguri au fost stabilite pe baza caracteristicilor tehnice ale senzorilor și sunt configurabile dinamic de operator prin interfața web.

3.6 Componenta Backend – FastAPI

Backend-ul este implementat în Python 3.13 cu framework-ul FastAPI, ales pentru performanța asincronă (bazat pe asyncio), generarea automată a documentației API (OpenAPI/Swagger) și suportul nativ pentru WebSocket. Componentele principale sunt:

- **main.py:** Punctul de intrare al aplicației. Gestionează starea globală a nodurilor, pornește loop-ul de simulare sau gateway-ul serial, inițializează watchdog-ul de offline și definește toate endpoint-urile REST și WebSocket.
- **models.py:** Modelele de date Pydantic pentru validare: SensorPacket, NodeState, NodeCommand, GasThresholds, Event, SystemStatus. Enum-urile NodeStatus, MessageType, Priority, GasLevel asigură validarea strictă a valorilor primite de la hardware.
- **risk_engine.py:** Implementarea motorului de evaluare a riscului. Funcțiile compute_risk_score(), classify_gas_level() și classify_risk() sunt pure (fără side-effects), ușor de testat și de extins.
- **routing.py:** Implementarea algoritmului Dijkstra pentru calculul rutelor optime. Funcțiile get_all_routes() și route_to_gateway() sunt utilizate atât pentru procesarea pachetelor cât și pentru endpoint-ul REST de vizualizare a rutelor.
- **simulator.py:** Generatorul de date sintetice pentru modul de simulare. Generează date realiste cu drift de temperatură, variații de gaz, discharge de baterie, și suportă scenarii de alertă, offline, pierdere pachete și mentenanță.
- **serial_gateway.py:** Clientul serial asincron pentru modul hardware. Citește linii JSON de pe portul serial, parsează pachetele cu fallback sigur pentru enum-uri necunoscute, și apelează callback-ul de procesare.
- **database.py:** Accesul la baza de date SQLite prin context manager db_cursor(). Tabele: events, sensor_history, settings, commands_log.

- **websocket_manager.py:** ConnectionManager pentru gestionarea conexiunilor WebSocket active. Broadcast asincron cu cleanup automat al conexiunilor moarte.

3.7 Interfața Web de Monitorizare

Interfața web este implementată în HTML5/CSS3/JavaScript vanilla (fără framework frontend), servită direct de FastAPI ca fișiere statice. Această alegere simplifică deployment-ul (un singur server) și elimină dependența de un bundler frontend. Componentele principale ale interfeței sunt:

- **Harta 2D a planului etajului:** Canvas HTML5 cu redare HiDPI (suport ecrane Retina). Afișează nodurile colorate după stare (verde/portocaliu/roșu/gri/violet), link-urile de comunicație (continue sau întrerupte), ruta activă a nodului selectat (linie galbenă), și scorul de risc afișat sub fiecare nod.
- **Carduri noduri senzori:** Pentru fiecare nod: temperatură, valoare gaz colorată după nivel, scor risc cu bară vizuală, nivel baterie și badge de stare.
- **Grafice live Chart.js:** Trei grafice în timp real cu istoricul ultimelor 30 de citiri: temperatură (°C), gaz (ADC 0–4095) și scor risc (0–100). Toate cele 4 noduri sunt afișate simultan cu culori distincte.
- **Panoul de detalii nod selectat:** La selectarea unui nod: toți parametrii senzori, ruta activă text, nivel baterie cu bară vizuală, RSSI, latență și tabelul complet al rutelor către toate destinațiile (mod normal vs. mod alertă cu costuri).
- **Jurnalul de evenimente:** Log live al ultimelor 80 de evenimente (alertă, warning, offline, comenzi). Fiecare eveniment afișează ora, zona și descrierea. Evenimentele noi apar instant prin WebSocket fără refresh.
- **Panoul de control simulare:** Butoane pentru: forțare alertă pe nod specific, simulare offline, simulare pierdere pachete, simulare cădere rută, mod mentenanță, reset complet sistem, reset nod individual.
- **Setări praguri gaz:** Modal de configurare a celor 5 praguri de clasificare gaz (normal_max, low_max, medium_max, high_max, critical_min) și activare/dezactivare buzzer la nivel HIGH. Modificările sunt persistate în SQLite și aplicate imediat.

4. Activități Desfășurate și Rezultate Obținute

4.1 Activități Realizate

Pe parcursul semestrului curent au fost finalizate cu succes toate componentele software ale sistemului. Activitățile principale realizate sunt:

- Studiul literaturii de specialitate: protocoale WSN, algoritmi de rutare, sisteme de detecție incendii, platforme IoT.
- Proiectarea arhitecturii complete: topologia rețelei, structura pachetelor de date, interfețele între componente.
- Implementarea firmware-ului C++ pentru nodurile senzori ESP32: module senzori, rutare Dijkstra, comunicație ESP-NOW, procesare comenzi.
- Implementarea firmware-ului C++ pentru gateway-ul ESP32: recepție ESP-NOW multi-nod, bridge serial JSON, procesare comenzi.
- Implementarea backend-ului Python/FastAPI: motor de risc, algoritm rutare, gestiune stări noduri, WebSocket broadcast, SQLite.
- Implementarea interfeței web: hartă 2D Canvas HiDPI, grafice live Chart.js, panou control, setări configurabile.
- Configurarea infrastructurii Raspberry Pi cu Python venv, uvicorn și tunel ngrok pentru acces remote.
- Testarea extensivă în modul simulare: scenarii normale, alertă, offline, pierdere pachete, mentenanță, rutare alternativă.
- Identificarea și remedierea problemelor de implementare apărute în procesul de testare, cu îmbunătățiri aduse robusteții componentelor de comunicație și procesare a datelor.

4.2 Rezultate Obținute

Testarea sistemului în modul de simulare a produs următoarele rezultate concrete:

- **Motorul de risc:** Scoruri consistente cu scenariile testate. Exemplu: $T=70^{\circ}\text{C}$ + gaz critic + 2 vecini în alertă → scor 97/100. $T=25^{\circ}\text{C}$ + gaz normal → scor 3/100. În testele cu parametri în plajele normale, sistemul nu a generat alarme false.
- **Rutare adaptivă:** Reconvergența rutei după căderea unui nod intermediar se produce în maximum 3 secunde (un ciclu de simulare). Schimbarea automată a coeficienților în modul alertă verificată funcțional.
- **Latența WebSocket:** Pe rețea locală (LAN): sub 200ms end-to-end. Prin tunel ngrok (internet public): 300–500ms. Reconectare automată după pierdere conexiune: 3 secunde.
- **Stabilitate:** Sistemul a rulat continuu mai multe ore fără repornire, gestionând corect scenariile de eroare testate.
- **Interfață web:** Testată pe Chrome, Firefox, Safari, Edge și pe dispozitive mobile (iOS/Android). Harta 2D redată corect la toate rezoluțiile, inclusiv ecrane Retina (HiDPI).

4.3 Provocări Întâmpinate și Soluțiile Adoptate

Pe parcursul implementării au fost întâmpinate mai multe provocări tehnice:

- **Compatibilitate Python 3.13 pe Raspberry Pi:** Raspberry Pi OS Bookworm include Python 3.13 implicit. Biblioteca pydantic 2.7.1 (fixată inițial) nu suportă Python 3.13 din cauza incompatibilității pyo3 cu versiunea respectivă. Soluție: actualizarea constrângerilor de versiune la `pydantic>=2.10.0`.
- **Redarea Canvas pe ecrane HiDPI:** Pe ecrane cu `devicePixelRatio>1`, funcțiile de desenare Canvas utilizau pixeli fizici după `scale()`, generând grid de 4× ori mai dens. Soluție: toate coordonatele se calculează în pixeli logici (`CANVAS_W/CANVAS_H`), `scale()` aplicat o singură dată la inițializare.
- **Propagarea excepțiilor prin serial gateway:** O excepție în callback-ul de procesare a pachetelor propaga prin `_read_loop` și cauza închiderea portului serial cu 5 secunde de date pierdute. Soluție: `try/except` separat pentru excepții de callback, distinct de excepțiile de I/O serial.

4.4 Activități în Curs de Finalizare

În etapa curentă se finalizează:

- Calibrarea fină a pragurilor de gaz pe baza măsurătorilor din mediul real de instalare
- Testarea extinsă a rutării adaptive în condiții variate de semnal radio
- Redactarea documentației tehnice complete a lucrării de diplomă
- Realizarea schemelor electrice finale și a diagramelor UML pentru anexe

5. Concluzii Preliminare

Lucrarea prezintă un sistem IoT distribuit complet funcțional pentru detectarea timpurie a incendiilor, cu arhitectură originală și contribuții clare față de soluțiile existente. Componenta software — firmware ESP32, backend FastAPI și interfață web — a fost implementată integral și validată extensiv prin testare în modul de simulare.

Contribuțiile principale ale lucrării sunt:

- Algoritm de rutare adaptivă cu comutare dinamică a funcției de cost în funcție de starea de alertă a rețelei, original față de soluțiile cu rutare statică din literatura de specialitate
- Motor de risc multi-parametric cu confirmare de la noduri vecine, reducând alarmele false față de sistemele cu senzor unic
- Arhitectură software completă open-source cu acces web remote, extensibilă pentru integrarea de noi tipuri de senzori sau protocoale de comunicație
- Mod de mentenanță distinct pentru noduri în service, care previne alarmele false și dezactivarea incorectă a monitorizării

Sistemul demonstrează fezabilitatea tehnică a unui sistem WSN de detecție a incendiilor cu cost redus (sub 25 EUR per nod) față de soluțiile comerciale existente (mii de EUR per detector), menținând capabilități avansate de detectare, rutare și monitorizare.

Pașii următori constau în integrarea hardware a senzorilor fizici și testarea în condiții reale, urmată de finalizarea documentației tehnice complete.

Bibliografie

- [1] Inspectoratul General pentru Situații de Urgență (IGSU), "Raport de activitate 2024," București, România, 2025.
- [2] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, E. Cayirci, "Wireless sensor networks: a survey," *Computer Networks*, vol. 38, no. 4, pp. 393–422, Mar. 2002.
- [3] Espressif Systems, "ESP-NOW User Guide v2.0," Technical Documentation, Espressif Systems, Shanghai, 2023. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- [4] Bosch Sensortec GmbH, "BME280 – Combined humidity and pressure sensor, Data sheet," Document BST-BME280-DS002, Rev. 1.6, Bosch, 2020.
- [5] Hanwei Electronics Co., "MQ-2 Semiconductor Sensor for Combustible Gas, Technical Data," Hanwei Electronics, Zhengzhou, China, 2018.
- [6] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [7] R. Sharma, A. Kumar, P. Gupta, "Wireless Sensor Network based Fire Detection System for Industrial Applications using ZigBee," *International Journal of Engineering Research and Technology*, vol. 4, no. 6, pp. 312–316, 2015.
- [8] X. Zhang, M. Li, H. Wang, "Multi-sensor data fusion for fire detection in wireless sensor networks," *IEEE Sensors Journal*, vol. 19, no. 8, pp. 3021–3030, Apr. 2019.
- [9] V. Caballero, D. Vernet, A. Zaballos, "Wildfire Detection System using Convolutional Neural Networks and IoT Sensors," *Sensors*, vol. 21, no. 19, p. 6473, 2021.
- [10] S. Misra, C. Reeves, T. Bhatt, "Wireless sensor networks for IoT-based fire detection: A survey," *IEEE Internet of Things Journal*, vol. 8, no. 10, pp. 7862–7875, 2021.
- [11] Espressif Systems, "ESP32 Technical Reference Manual v5.0," Espressif Systems, Shanghai, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf